Structural Ambiguity in BPE Tokenization: From Vocabulary Merges to Attention Collapse

Dayna Blackwell, Blackwell Systems ·
dayna@blackwell-systems.com

2026-06-25 · DOI: 10.5281/zenodo.20789620

Contents

Structural Ambiguity in BPE Tokenization: From Vocabulary Merges to Attention Collapse

Dayna Blackwell, Blackwell Systems · dayna@blackwell-systems.com

Date: 2026-06-25 · **DOI:** 10.5281/zenodo.20789620

Abstract

JSON is the dominant data interchange format in large language model (LLM) tool ecosystems, yet its structural grammar interacts poorly with Byte-Pair Encoding (BPE) tokenizers at the subword level. We present a systematic cross-tokenizer analysis of JSON's structural boundaries across 43 tokenizers from 20 providers, covering every major LLM family in production.

We find that the most common JSON field names (including `id`, `name`, `type`, `value`, `title`, `time`, `text`, `url`, `path`, and `description`) exist as merged vocabulary entries where the opening quote fuses with the field name on 28-30% of tested tokenizers. These merges are hardcoded dictionary entries (e.g., `"name` is entry #32586 in GPT-4's cl100k vocabulary) and are deterministic, not context-dependent. GPT-4 contains 117 such quote+letter vocabulary entries; Claude and Gemma contain 2-3. An exhaustive vocabulary scan across all 43 tokenizers reveals that the pipe delimiter has 24 mergeable words in any vocabulary. JSON's three structural characters have a combined adversarial surface of 707 words (quote 193, colon 232, comma 282). The tab has 1,238.

On real evaluation data (29,025 checks across 43 tokenizers), JSON exhibits a boundary merge rate of 8.17% compared to 0.47% for a comparison format using pipe delimiters and 32.91% for a tab-delimited format. We further show that JSON overhead reaches 81% at 500 rows (52% repeated field names, 29% structural characters), growing O(n) per row, while header-factored formats maintain O(1) overhead. A grammar swap experiment across 5 delimiter sets and 800 measurements confirms that token savings are structural (0.4 percentage point spread), not delimiter-specific.

We present a structural equivalence proof showing that pipe delimiters maintain 99.5% grammar isolation across all 43 tokenizers (@ 100%, < 100%, | 99.2%), while JSON grammar fuses into multi-operation tokens on 43/43 tokenizers (92.5% of quote-containing tokens). Attention mechanism analysis on Pythia 410M and Gemma 2B reveals that JSON attention entropy crosses the comparison format at 50 orders, and JSON grammar attention collapses from 30% to 8.6% at scale.

These findings are irrecoverable: tokenizer vocabularies are frozen post-training, all model weights depend on them, and tokenization occurs before the transformer processes input. We present correlation evidence from 2,400+ LLM evaluations showing JSON comprehension drops to 53.4% at 500-record scale where structural ambiguity, attention dilution, and grammar attention collapse compound.

Keywords: BPE tokenization, JSON, structural ambiguity, vocabulary merge, LLM comprehension, wire format, attention dilution, structural equivalence

1. Introduction

When structured data enters an LLM's context window, it passes through a tokenizer that converts the character sequence into a sequence of integer IDs. The tokenizer is trained separately from the model and uses a fixed vocabulary. Different model families use different tokenizers trained on different corpora. This creates a problem: the same structured data can produce different token sequences on different models.

For natural language, tokenizer variance is well understood and largely harmless. Whether "understanding" is one token or two, the model reads the same characters. But for structured data formats like JSON, tokenizer variance has a more consequential effect: the characters that mark structural boundaries (quotes, colons, braces) can merge with payload content, making field boundaries invisible at the token level.

This paper presents a mechanistic analysis of how and why JSON's structural grammar breaks down under BPE tokenization. We distinguish between two types of content in any structured format: grammar symbols (delimiters that define structure) and payload content (the actual data values). When grammar symbols merge with payload content into single tokens, we call this a "boundary merge." The resulting token conflates structural markup with semantic content, forcing the model to decompose structure from within a single embedding rather than reading it from token boundaries.

Prior work has noted JSON's token overhead in passing. Deekeswar (2024) measured that 1,000 JSON records consume approximately 80,000 tokens with the majority being repeated keys and punctuation. Karim and Batatia (2025) explored structured tokenization for LLM training data. However, no prior work has performed a systematic mechanistic analysis of exactly how and where JSON's structure breaks down at the BPE level, which tokenizers are affected, and whether the problem is recoverable.

We fill this gap with six contributions:

1. An exhaustive vocabulary scan of 43 tokenizer dictionaries identifying all entries where delimiter characters fuse with alphabetic content, with the complete adversarial surface for pipe (24 words), quote (193 words), and tab (1,238 words).
2. A cross-tokenizer boundary merge analysis on real evaluation data (29,025 checks), quantifying the rate at which JSON's structural delimiters merge with field names.
3. A structural equivalence proof showing that pipe-delimited formats maintain deterministic grammar isolation (99.5%) while JSON grammar fuses on all 43 tokenizers.
4. A grammar swap experiment proving that token savings from header-factored formats are structural properties, not artifacts of specific delimiter choices.
5. Attention mechanism analysis on Pythia 410M and Gemma 2B, demonstrating entropy crossover and grammar attention collapse as the transformer-level mechanism behind comprehension failures.

6. Correlation evidence from 2,400+ LLM evaluations connecting tokenization mechanics to observed comprehension outcomes.

We use Graph Compact Format (GCF), a header-factored wire format with pipe delimiters, as the comparison format throughout. GCF was selected because its grammar characters are drawn from a set empirically verified to have near-zero merge rates across all tested tokenizers.

---

2. Background

2.1 BPE Tokenizers

Modern LLMs use Byte-Pair Encoding (BPE) tokenizers (Sennrich et al., 2016) trained on large text corpora. BPE builds a vocabulary by iteratively merging the most frequent byte sequences in the training data. The result is a fixed lookup table mapping strings to integer IDs. GPT-4's cl100k vocabulary contains 100,256 entries; Gemma 2's contains 256,128.

At inference time, the tokenizer greedily matches the longest vocabulary entry at each position in the input. If the string `"name` exists as vocabulary entry #32586, the tokenizer will always select it as a single token rather than splitting it into `"` (#1) + `name` (#609). This is deterministic. There is no probability or context-dependence in the selection; if the entry exists, it wins.

2.2 Grammar vs. Payload Distinction

Any structured data format contains two types of content:

- **Grammar symbols** (delimiters, structural markers): characters that define where fields start and end. A format designer controls these.
- **Payload content** (data values): the user's actual data. A format designer cannot control how these tokenize without altering the data itself.

This distinction matters for tokenizer analysis because grammar symbols repeat on every row (e.g., `"fieldName":` appears 500 times in a 500-element JSON array), while payload content varies. When grammar symbols merge with payload content, the resulting ambiguity compounds linearly with data size.

2.3 Structural vs. Semantic Equivalence

Two types of cross-tokenizer equivalence are relevant:

- **Structural equivalence**: all models see field boundaries at the same token positions. They agree on WHERE the structure is.
- **Semantic equivalence**: all models see the same data values. They agree on WHAT the content is.

Semantic equivalence is always preserved regardless of format. Whether `userName` is one token or two, the model reads the same characters. Structural equivalence is the critical property. If models disagree on where fields start and end, they are parsing different structures from the same input. This is the mechanism that produces model-dependent comprehension failures.

---

3. Methodology

3.1 Tokenizers Tested

We tested 43 tokenizers from 20 providers, covering every major LLM family in production. This is the most comprehensive tokenizer boundary analysis published for any wire format. Vocabulary sizes range from 32K (Mistral 7B, LLaMA 2, Arctic) to 262K (Gemma 3). Representative tokenizers:

Provider	Tokenizers	Models Covered
OpenAI	cl100k_base, o200k_base, GPT-2	GPT-4, GPT-4o, GPT-5.x
Anthropic	Claude tokenizer	Claude 3.5, 4.x
Meta	LLaMA 2, LLaMA 3, LLaMA 3.1, CodeLlama, TinyLlama	LLaMA family
Google	Gemma 2, Gemma 3, T5	Gemma family
Mistral	7B v0.1, 7B v0.3, Nemo, Mixtral, Codestral	Mistral/Mixtral family
Alibaba	Qwen 2, Qwen 2.5, Qwen 2.5 Coder, Qwen 3, QwQ	Qwen family
DeepSeek	V2 Lite, V3, R1, Coder V2 Lite	DeepSeek family
Microsoft	Phi-2, Phi-3, Phi-4	Phi family
TII	Falcon 7B, 40B, 2 11B	Falcon family
01.AI	Yi 1.5, Yi Coder	Yi family
BigCode	StarCoder2 7B, 15B	StarCoder family
Others	Nemotron (NVIDIA), Jamba (AI21), StableLM (Stability), Pythia (EleutherAI), Arctic (Snowflake), OLMo (AllenAI), Marco-o1 (Alibaba AIDC)	Various

These tokenizers were each trained on different corpora (the large text datasets used to learn which byte sequences to merge into tokens) with different merge priorities. Their disagreements on how to tokenize the same input reveal fundamental properties of that input's structure. Initial experiments were conducted on 8 representative tokenizers using JavaScript-based tooling; the full 43-tokenizer analysis was conducted using Python with HuggingFace tokenizer libraries.

3.2 Measurements

We measured nine properties:

1. **Boundary merge rate**: For each format, we tested all combinations of field names and values across all 43 tokenizers (29,025 checks for GCF, 1,935 for JSON). A boundary merge occurs when a structural delimiter (quote in JSON, pipe in GCF, tab in TOON) fuses with adjacent content into a single token.
2. **Common field merge analysis**: We tested 45 common field names from production APIs across all 43 tokenizers to identify which fields merge and at what rate (scripts: `common-field-merge-analysis.mjs`, `hf-tokenizer-analysis.py`).
3. **Exhaustive vocabulary scan**: We decoded every entry in each of the 43 tokenizer vocabularies (32K to 262K entries each) and classified entries containing delimiter characters fused with alphabetic content. This produces the complete adversarial surface: every word that can ever merge with each delimiter (scripts: `vocabulary-full-scan.mjs`, `adversarial-vocab-dump.py`).
4. **Token overhead analysis**: We measured the proportion of tokens consumed by repeated field names, structural characters, and actual data values at scales from 10 to 1,000 rows (script: `json-tokenization-analysis.mjs`).
5. **Grammar swap experiment**: We replaced all GCF delimiters with 4 alternative sets (all drawn from the non-merging character set) and re-measured savings across 5 payload types, 4 sizes, and 8 tokenizers (800 measurements total; script: `grammar-swap-experiment.mjs`).
6. **Token savings consistency**: We measured GCF vs. JSON savings across all 43 tokenizers at scales from 10 to 500 records to verify cross-tokenizer stability (scripts: `hf-tokenizer-analysis.py`, `tokenizer-variance.mjs`).
7. **Structural equivalence proof**: We tokenized a realistic multi-section payload across all 43 tokenizers and checked every token containing a row-level grammar symbol to determine isolation rates (script: `structural-equivalence-proof.py`).
8. **Attention mechanism analysis**: We loaded Pythia 410M (24 layers, 16 heads) and Gemma 2B (26 layers, 8 heads) and extracted attention weights from every layer and head while processing identical data in GCF vs JSON at increasing scale (script: `attention-analysis.py`).
9. **Grammar attention classification**: We classified every token as grammar or payload and measured what fraction of the model's total attention goes to each category across scales from 5 to 100 records.

3.3 Evaluation Data

Comprehension correlation data comes from 2,400+ LLM evaluation calls across 11 models and 3 providers (Anthropic, OpenAI, Google), using both standard workloads (500 orders, nested data) and structurally complex payloads (500 symbols, 200 edges, 13 questions per run). These evaluations are described in the companion paper "GCF: A Token-Optimized Wire Format for Structured LLM Interactions" (DOI: 10.5281/zenodo.20579817); we reference the results here to connect tokenization mechanics to observed comprehension outcomes.

3.4 Field Name Frequency Data

To establish that the affected field names are representative of real-world usage, we reference the Web Data Commons JSON-LD dataset (University of Mannheim, 2024), which analyzed JSON properties across 2.39 billion web pages. In that dataset, `name` is the #1 most common JSON property (3.5 billion occurrences) and `url` is #2 (2.6 billion occurrences). Both merge with the opening quote on 50-63% of tested tokenizers.

---

4. Results

4.1 JSON Field Boundaries Tokenize Inconsistently

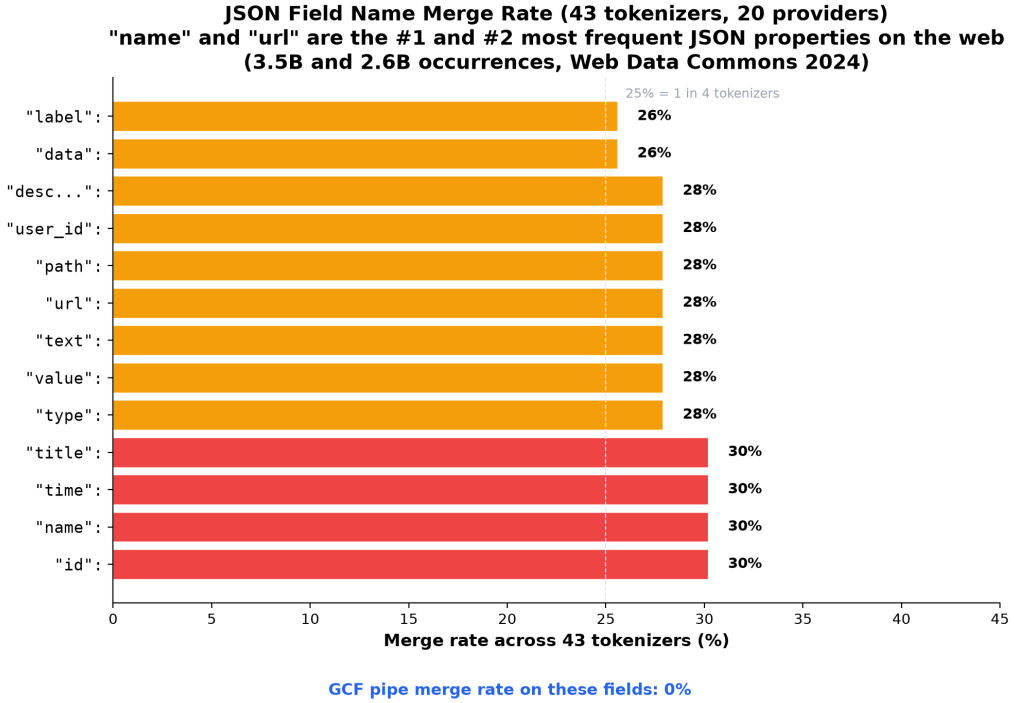

Figure 1: Field merge rates across 43 tokenizers

The most common JSON field names in computing merge with the opening quote on roughly 30% of all 43 tested tokenizers:

Field Pattern	Merge Rate	Affected Tokenizer Families
`"id":`	30.2% (13/43)	GPT-4, GPT-4o, LLaMA 3.x, Qwen, Phi-4, StableLM, Marco, Mistral Nemo
`"name":`	30.2% (13/43)	Same group
`"time":`	30.2% (13/43)	Same group
`"title":`	30.2% (13/43)	Same group
`"type":`	27.9% (12/43)	Same group minus Mistral Nemo
`"value":`	27.9% (12/43)	Same group minus Mistral Nemo
`"url":`	27.9% (12/43)	Same group minus Mistral Nemo
`"text":`	27.9% (12/43)	Same group minus Mistral Nemo
`"path":`	27.9% (12/43)	Same group minus Mistral Nemo
`"user_id":`	27.9% (12/43)	Same group minus Mistral Nemo
`"description":`	27.9% (12/43)	Same group minus Mistral Nemo

According to the Web Data Commons dataset (University of Mannheim, 2024), `name` is the #1 most common JSON property on the web (3.5 billion occurrences across 2.39 billion pages) and `url` is #2 (2.6 billion occurrences). Both merge on 28-30% of tokenizers.

At 500 rows with `id`, `name`, and `type` fields, this produces approximately 1,500 field boundaries where nearly a third of all models see a hidden merge. The merge is deterministic per tokenizer: it either always occurs or never occurs for a given field name, because it depends solely on whether the merged string exists in the vocabulary.

Maximum tokenization variance Searching across 840 JSON field+value patterns (40 field names x 21 values), the maximum variance case is `"userName":"req_xyz789"`, which produces 7 distinct tokenizations across 8 models:

```
GPT-4, LLaMA:     ["][userName][":"][req][_xyz][789]["]
GPT-4o:           ["user][Name][":"][req][_xyz][789]["]
Claude:           ["][userName][":"][req][_][xyz][789]["]
Qwen 2.5:         ["][userName][":"][req][_xyz][7][8][9]["]
DeepSeek V3:      ["][user][Name][":"][req][_][xyz][789]["]
Gemma 2:          ["][userName][":"][req][_][xyz][7][8][9]["]
Mistral Nemo:     ["][user][Name][":"][req][_x][yz][7][8][9]["]
```

A complete JSON object `{"orderId":"ORD-001","value":"shipped"}` produces 4 different token counts (12, 13, 14, 15) depending on the model. The same data is literally a different length

on different model families, affecting attention patterns, positional encodings, and context budget.

4.2 The Merge Mechanism

The variance described above has a specific cause: BPE merging absorbs the opening quote into the field name. When GPT-4's tokenizer encounters `"value":"pending"`, it produces:

```
["value] [":"] [pending] ["]    (4 tokens)
```

Claude's tokenizer produces:

```
["] [value] [":"] [pending] ["]    (5 tokens)
```

The structural boundary (where the field name starts) is at a different token position depending on which model processes the data. On GPT-4, the opening quote is fused with the content. On Claude, it is separate. The model must learn to decompose the merged token `"value` into "opening quote followed by a field name" rather than treating it as a single semantic unit.

4.3 Boundary Merge Rates on Real Data

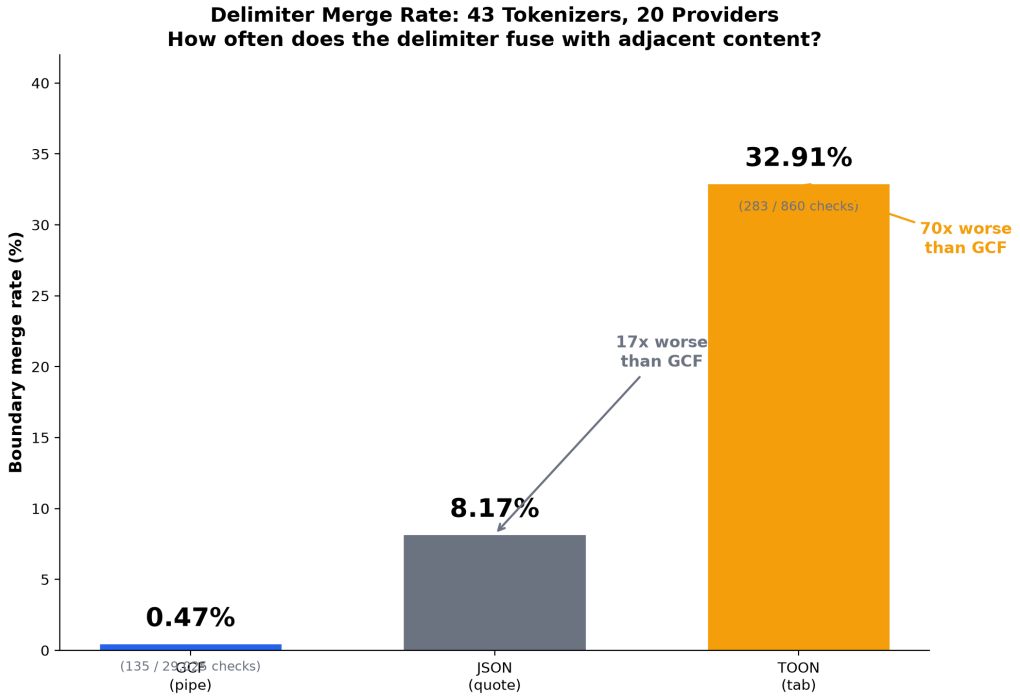

Figure 2: Delimiter merge rates: pipe 0.47%, quote 8.17%, tab 32.91%

On real evaluation data across all 43 tokenizers:

Format	Boundary Merges	Total Checks	Merge Rate	Primary Cause
GCF (pipe)	135	29,025	**0.47%**	Only `|cancelled` on 3 tokenizers (Mistral Nemo, DeepSeek V3, DeepSeek R1)
JSON (quote)	158	1,935	**8.17%**	`"id":`, `"name":`, `"type":` merge on 13/43 tokenizers
TOON (tab)	283	860	**32.91%**	Tab merges on GPT-4o (100%), GPT-4 (95%), LLaMA 3 (95%), Qwen (95%)

GCF exhibits 94.3% fewer boundary merges than JSON on real data, and JSON in turn merges far less than TOON.

The critical difference in scaling behavior: JSON's merges are caused by field names, which repeat on every row and compound linearly. GCF's merges are caused by one specific value (`cancelled`) on 3 tokenizers. At 500 rows with `"id"` and `"name"` fields, JSON has approximately 500 hidden boundaries. GCF has zero on field names.

All GCF grammar characters encode as exactly 1 token on all 43 tokenizers:

```
value|pending     -> [value][|][pending]     ALL 43 tokenizers
name|Alice        -> [name][|][Alice]        ALL 43 tokenizers
orderId|ORD-001   -> [orderId][|][ORD][-][001]  ALL 43 tokenizers
```

4.4 Root Cause: Vocabulary Entries

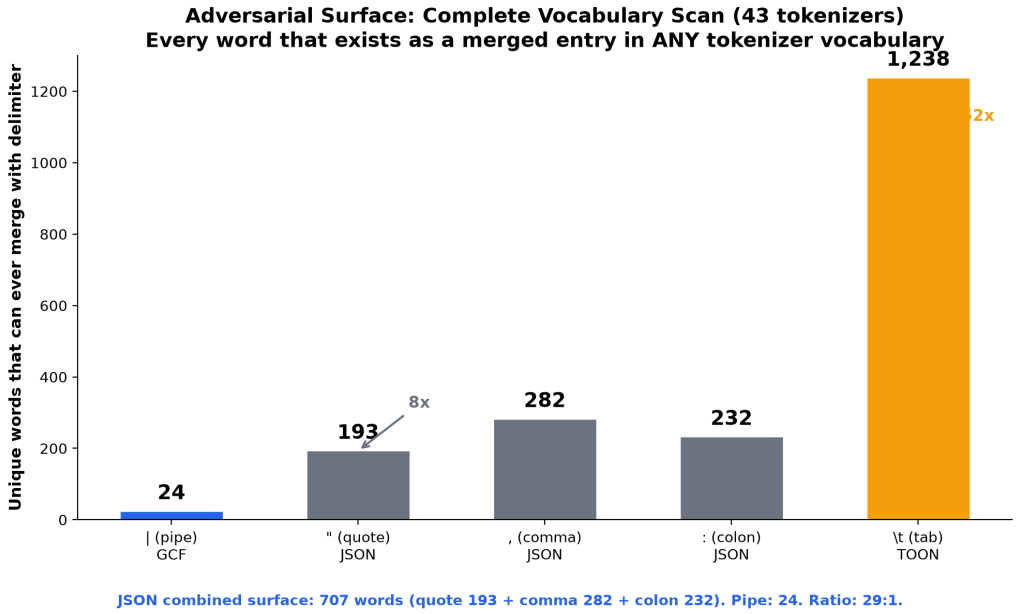

Figure 3: Vocabulary merge entries by tokenizer

The merge behavior is not a context-dependent tokenizer decision. It is a dictionary lookup. We decoded every entry in each of the 43 tokenizer vocabularies and classified entries containing delimiter characters fused with alphabetic content:

Tokenizer	Vocab Size	Quote+Letter	Pipe+Letter	Tab+Letter	Multi-Grammar
GPT-4 (cl100k)	~100K	**117**	22	**1,173**	874
GPT-4o (o200k)	~199K	**108**	8	**1,036**	735
Claude	~65K	**3**	0	0	667
LLaMA 2	~32K	0	0	0	122
LLaMA 3/3.1	~128K	**153**	277	0	881
Qwen 2.5	~152K	**154**	40	0	874
DeepSeek V3/R1	~129K	**48**	5	0	287
Gemma 2	~256K	**4**	0	0	735
Gemma 3	~262K	**2**	0	0	861
Mistral Nemo	~131K	**38**	3	0	415
Phi-4	~100K	**153**	116	0	874

Tokenizer	Vocab Size	Quote+Letter	Pipe+Letter	Tab+Letter	Multi-Grammar
Falcon	~65K	4	1	0	92
StarCoder2	~49K	3	2	0	535
Jamba	~66K	1	**1,543**	0	113

GPT-4 has 1,173 tab+letter vocabulary entries. Tab-separated data was so common in the training corpora (TSV files, log output, terminal formatting) that the tokenizer absorbed tabs more aggressively than any other delimiter. No other non-OpenAI tokenizer has tab+letter entries. TOON chose the single worst delimiter for the two most widely deployed tokenizers.

Complete adversarial surface BPE is deterministic. If `|foo` exists as a vocabulary entry, the tokenizer will select it as a single token. If it does not exist, `|` and `foo` must be separate tokens. The definitive answer to "what can merge with each delimiter" is an exhaustive dump of every vocabulary entry starting with that delimiter across all 43 tokenizers:

Delimiter	Unique Mergeable Words	Used by	Examples
`|` (pipe)	**24**	GCF	`|null`, `|string`, `|required`, `|max`, `|min` (TypeScript unions)
`"` (quote)	**193**	JSON	`"name`, `"id`, `"type`, `"value`, `"text`, `"url`, and 187 others
`:` (colon)	**232**	JSON	`::self`, `::string`, `::id`, `::int` (C++/Rust scope resolution)
`,` (comma)	**282**	JSON	`,name`, `,value`, `,type` (CSV-style patterns in training data)

Delimiter	Unique Mergeable Words	Used by	Examples
`\t` (tab)	**1,238**	TOON	Nearly every common English word on GPT-4/4o

JSON uses three structural characters with a combined adversarial surface of **707 unique mergeable words** (quote 193 + colon 232 + comma 282). The pipe has 24. That is a 29:1 ratio. LLaMA 3.1 alone has 242 comma+letter and 198 colon+letter vocabulary entries. The entries `|name`, `|id`, `|type`, `|value`, `|status`, `|title` do not exist as vocabulary entries on any of the 43 tokenizers. The pipe will never merge with any common field name. This is not a statistical claim from sampling; it is a dictionary fact.

Specific token IDs These are actual dictionary entries with specific IDs, not hypothetical merges:

Pattern	GPT-4	GPT-4o	LLaMA	Qwen	DeepSeek	Mistral	Claude	Gemma
`"id`	#29800	#60094	#29800	#28700	–	#117579	–	–
`"name`	#32586	#74800	#32586	#31486	–	#117753	–	–
`"type`	#45570	#91290	#45570	#44470	–	–	–	–
`"value`	#64407	#180654	#64407	#63307	–	–	–	–
`"time`	#33239	#74035	#33239	#32139	–	#79174	–	–
`"title`	#83827	#187286	#83827	#82727	–	#110760	–	–
`"text`	#67351	#171858	#67351	#66251	–	–	–	–
`"url`	#61360	#124415	#61360	#60260	–	–	–	–
`"path`	#71788	#184610	#71788	#70688	–	–	–	–
`"description`	#69093	#150676	#69093	#67993	–	–	–	–
`"user`	#77622	#167975	#77622	#76522	–	–	–	–

Cross-verified: encoding `"name":"Alice"` with GPT-4's tokenizer confirms token #32586 is selected. The entries are active, not dead vocabulary.

Multi-grammar vocabulary entries Some vocabulary entries contain multiple JSON grammar symbols fused together:

Token	Grammar Characters	Present In
`":"`	`" : "`	8/8 tokenizers
`","`	`" , "`	8/8 tokenizers
`{"`	`{ "`	8/8 tokenizers
`":{"`	`" : { "`	8/8 tokenizers
`":["`	`" : [ "`	6/8 tokenizers
`"},`	`" } ,`	8/8 tokenizers
`"],`	`" ] ,`	8/8 tokenizers
`},{"`	`} , { "`	6/8 tokenizers

The token `":{"` exists on all 8 tokenizers. It represents four structural operations in one token: close a string, start a key-value pair, open an object, start a new string.

Pipe merge entries The pipe has a small number of merged entries (17 on GPT-4), but exclusively with programming keywords from type union syntax: `|null`, `|string`, `|max`, `|min`, `|required`. The entries `|name`, `|id`, `|type`, `|value` do not exist in any tested vocabulary. The pipe merges with type-system keywords, not with the field names that matter for structured data comprehension.

4.5 JSON Token Overhead

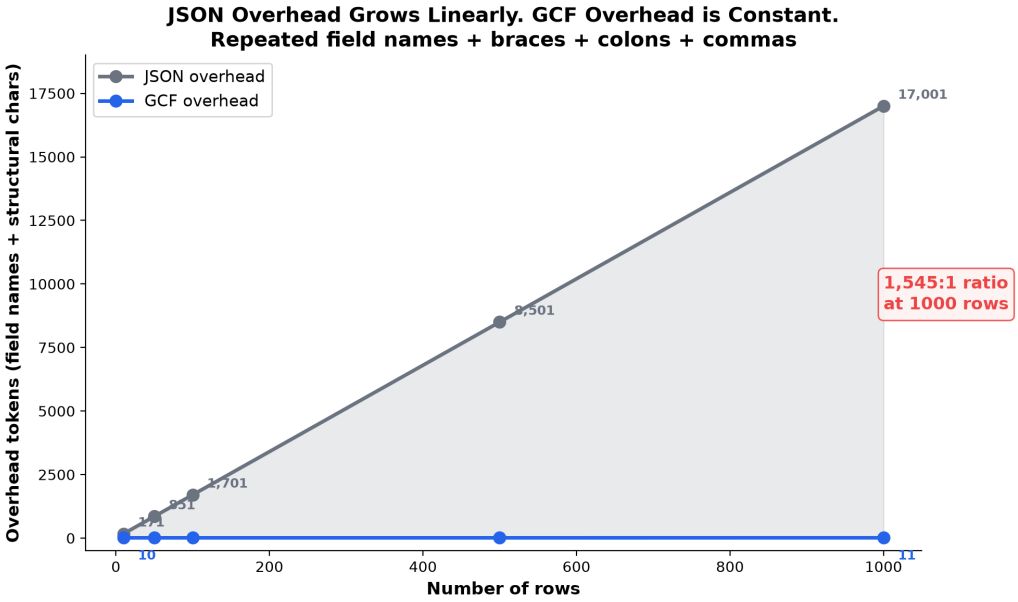

Figure 4: JSON overhead scaling: O(n) per row vs O(1) for header-factored formats

Beyond structural ambiguity, JSON consumes the majority of its tokens on content that carries zero information after the first row.

Token distribution at 500 rows (4-field frequency table)

Category	JSON Tokens	% of Total
Repeated field names (`"field":`, `"value":`, etc.)	5,500	52.4%
Structural characters (`{`, `}`, `[`, `]`, `:`, `,`)	3,001	28.6%
Actual data values	1,995	19.0%
Total	**10,496**	

Eighty-one percent of JSON's tokens are overhead. Only 19% carry actual information.

GCF for the same data:

Category	GCF Tokens	% of Total
Header (field names, declared once)	10	0.2%
Data rows	6,500	99.8%
Total	**6,510**	

Overhead scaling

Rows	JSON Overhead	GCF Overhead	Ratio
10	171 tokens	10 tokens	17:1
50	851 tokens	10 tokens	85:1
100	1,701 tokens	10 tokens	170:1
500	8,501 tokens	10 tokens	850:1
1,000	17,001 tokens	11 tokens	1,545:1

JSON overhead grows O(n) per row. Each additional row adds approximately 17 tokens of overhead. GCF overhead is O(1), constant regardless of row count. At 1,000 rows, the ratio is 1,545:1.

Cross-tokenizer validation

Tokenizer	JSON Tokens	GCF Tokens	Savings	JSON Field-Name Overhead
Claude (Anthropic)	10,996	7,013	36.2%	54.6%
GPT-4 (cl100k)	10,494	6,508	38.0%	52.4%
GPT-4o (o200k)	10,494	6,508	38.0%	52.4%
LLaMA 3.1 (Meta)	10,494	6,508	38.0%	52.4%
Qwen 2.5 (Alibaba)	13,150	9,166	30.3%	41.8%
DeepSeek V3	10,494	6,509	38.0%	57.2%
Gemma 2 (Google)	14,149	9,669	31.7%	42.4%
Mistral Nemo	13,649	9,167	32.8%	44.0%

Every tokenizer confirms that JSON spends 42-57% of its tokens on repeated field names alone.

4.6 Token Savings Consistency

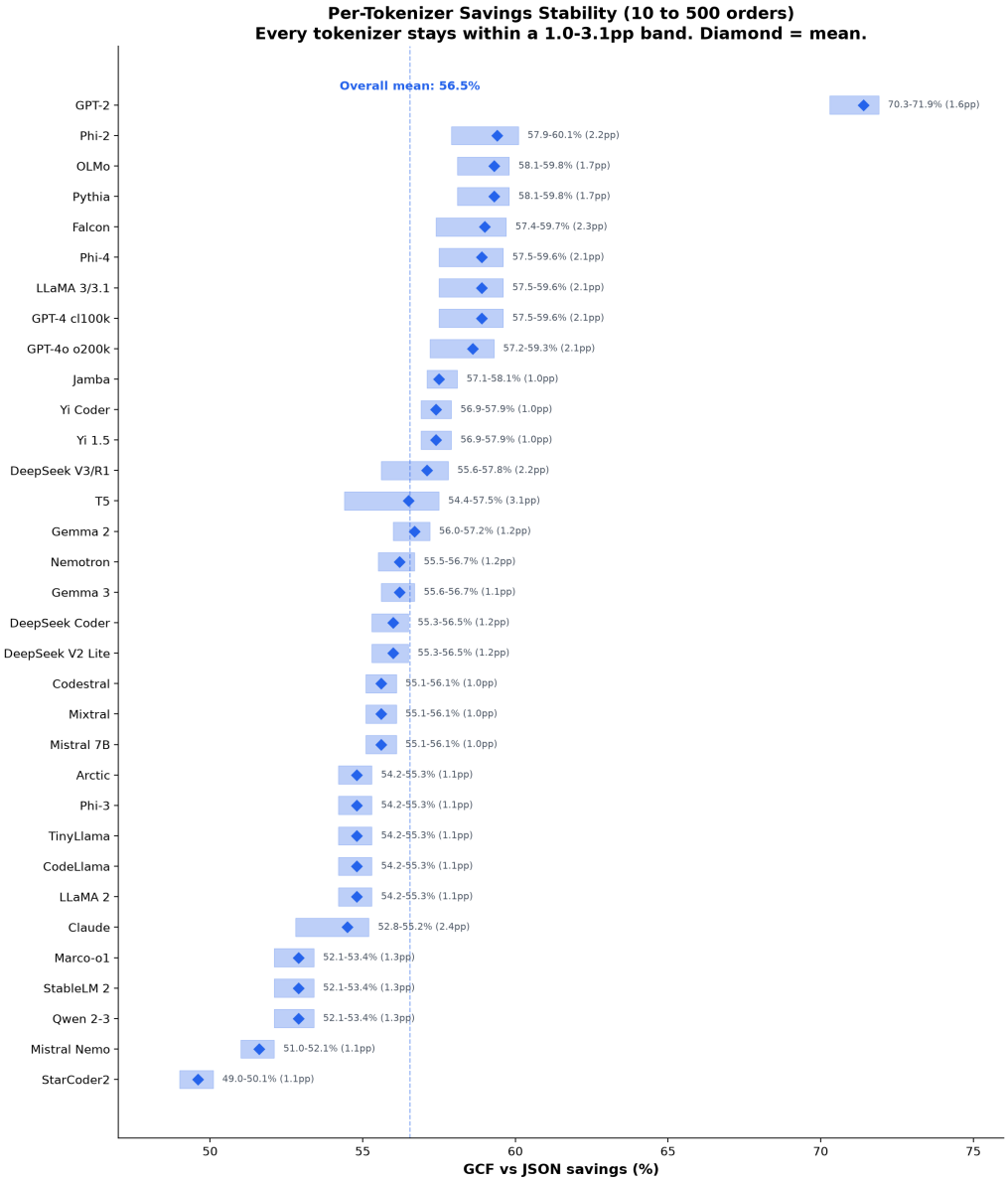

Figure 5: Savings stability bands across 43 tokenizers

Across all 43 tokenizers, GCF achieves 49-72% savings vs JSON (mean 56%) and 18-43% vs TOON (mean 28%) at 500 records. Representative results:

Tokenizer	GCF Tokens	JSON Tokens	Savings
GPT-4 (OpenAI cl100k)	40,190	98,148	**59.1%**
GPT-4o (OpenAI o200k)	40,591	98,548	**58.8%**
Claude (Anthropic)	43,704	96,719	**54.8%**
LLaMA 3.1 (Meta)	40,190	98,148	**59.1%**

Tokenizer	GCF Tokens	JSON Tokens	Savings
Qwen 2.5 (Alibaba)	52,402	109,468	**52.1%**
DeepSeek V3	43,694	102,348	**57.3%**
Gemma 2 (Google)	54,906	124,719	**56.0%**
Mistral Nemo	55,407	112,968	**51.0%**
Phi-4 (Microsoft)	40,190	98,148	**59.1%**
Falcon 7B (TII)	44,649	109,599	**59.3%**
Yi Coder (01.AI)	58,867	136,668	**56.9%**
StarCoder2 (BigCode)	57,356	112,419	**49.0%**

The worst case (StarCoder2, 49.0%) still nearly halves the token count. Savings remain stable across scales: each tokenizer stays within a 1.0-3.1pp band from 10 to 500 records. The mean holds at 55-57% vs JSON and 27-28% vs TOON across all 43 tokenizers at every scale tested.

Delimiter character safety analysis Before choosing structural delimiters, we tested all 94 printable ASCII characters (codes 33-126) across tokenizers on two criteria: (1) whether the character encodes as exactly 1 token in isolation, and (2) whether it merges with adjacent text. 74 of 94 characters are safe. 20 characters merge, including `.` (merges into `.validate`, `.com`), `-` (merges into `-token`, `-based`), `_` (merges into `_name`, `_id`), `/` (merges into `/api`, `/path`), and lowercase letters (common subword prefixes). JSON uses `.`, `"`, and `:` as structural characters. All three can create merge patterns. The pipe-delimited comparison format uses only characters from the safe set (`|`, `@`, `<`, `#`, `{`, `}`, `[`, `]`, `,`, `\n`).

4.7 Grammar Swap Experiment

To isolate whether savings are a structural property (header factorization, positional encoding) or an artifact of specific delimiter choices, we substituted all GCF delimiters with 4 alternative sets:

Set	Field	ID	Edge	Section	Schema
GCF (actual)	`|`	`@`	`<`	`##`	`{,}`
Alt A	`~`	`$`	`>`	`%%`	`(;)`
Alt B	`^`	`!`	`=`	`&&`	`{:}`
Alt C	`` ` ``	`#`	`~`	`!!`	`[|]`
Alt D	`;`	`%`	`^`	`$$`	`{+}`

Results across 5 payload types, 4 sizes, 8 tokenizers (800 measurements):

Delimiter Set	Overall Savings
GCF (actual)	60.6%
Alt A	60.3%
Alt B	60.7%
Alt C	60.3%
Alt D	60.4%

Total spread: 0.4 percentage points. Replacing every delimiter in the grammar produces the same compression. The efficiency is a mathematical property of eliminating key repetition, not an artifact of which characters are used.

Per-tokenizer consistency:

Tokenizer	GCF	Alt A	Alt B	Alt C	Alt D
Claude	60.8%	60.4%	60.8%	60.4%	60.4%
GPT-4	63.2%	62.8%	63.2%	62.8%	62.8%
GPT-4o	63.1%	63.1%	63.5%	63.1%	63.1%
LLaMA 3.1	63.2%	62.8%	63.2%	62.8%	62.8%
Qwen 2.5	56.3%	55.9%	56.3%	55.9%	55.9%
DeepSeek V3	63.4%	62.9%	62.9%	62.9%	63.7%
Gemma 2	60.2%	59.9%	60.2%	59.9%	59.9%
Mistral Nemo	56.0%	56.0%	56.5%	56.0%	56.0%

Variation per tokenizer across delimiter sets: 0.0-0.8 percentage points.

4.8 Structural Equivalence Proof

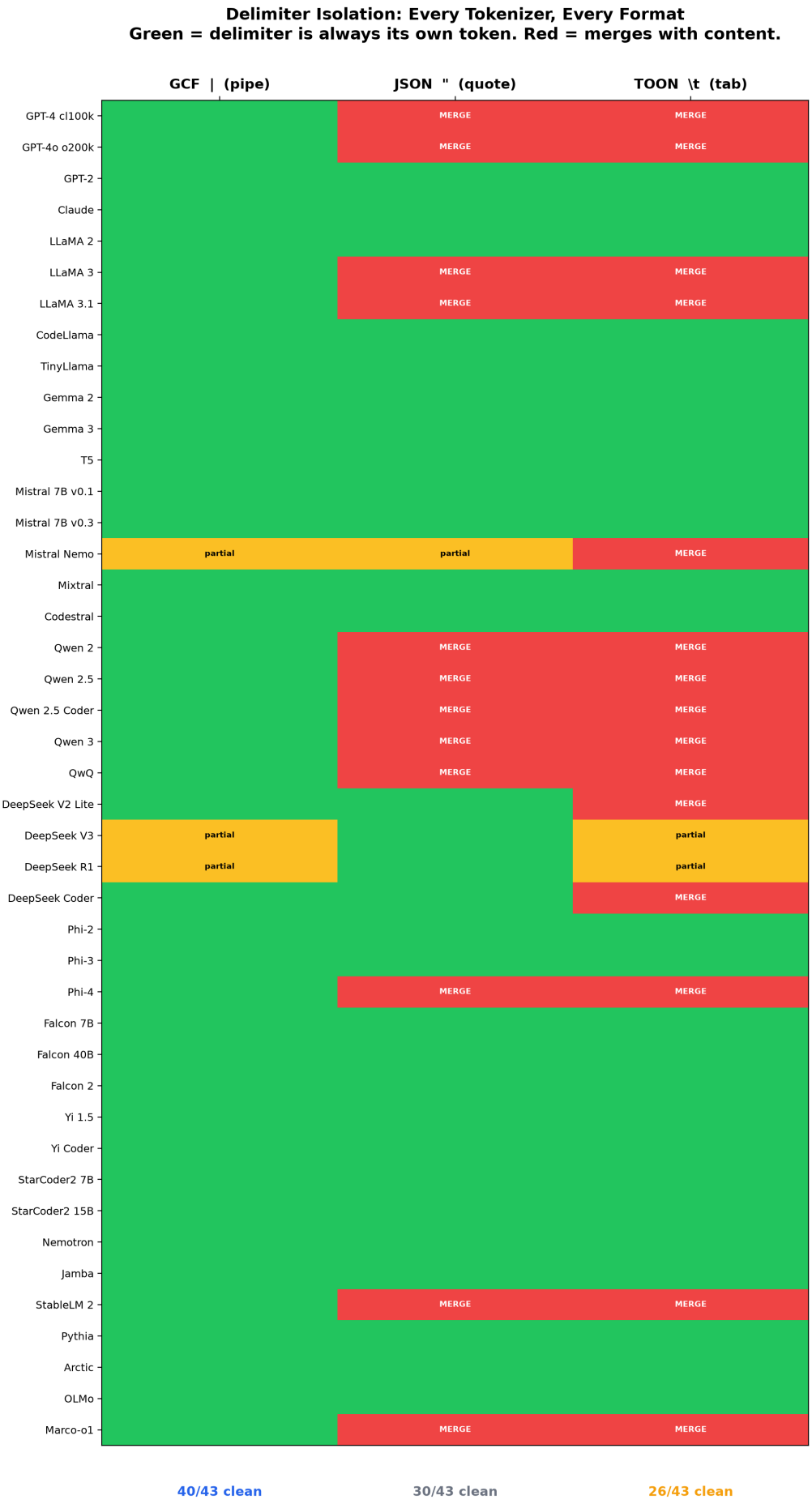

Figure 6: Structural equivalence: pipe 99.5% isolation vs JSON 7.5%

Parts 4.1-4.7 measured merge rates, vocabulary entries, and overhead. This section asks the definitive question: is the pipe-delimited grammar deterministic? When you tokenize a payload on any production tokenizer, does every grammar symbol remain its own token?

We tokenized a realistic multi-section payload (5 orders + 3 edges) across all 43 tokenizers and checked every token containing a row-level grammar symbol:

Symbol	Purpose	Total Tokens	Isolated	Merged	Isolation Rate
`@`	Symbol ID prefix	258	258	0	**100.0%**
`<`	Edge direction	126	126	0	**100.0%**
`|`	Field delimiter	774	768	6	**99.2%**
Overall		**1,158**	**1,152**	**6**	**99.5%**

The 6 pipe exceptions are all `|c` in the value `cancelled` on 3 tokenizers (Mistral Nemo, DeepSeek V3, DeepSeek R1). Pipe never merges with any field name on any tokenizer.

For comparison, JSON grammar tokens on the same data:

92.5% of quote-containing tokens fuse multiple grammar operations into a single token on **all 43 tokenizers**:

Token	Grammar Operations Fused	Present On
`":"`	Close string + colon + open string	42/43
`","`	Close string + comma + open string	41/43
`{"`	Open object + open string	42/43
`":`	Close string + colon	43/43
`,"`	Comma + open string	41/43

The token `":"` represents three structural operations packed into a single integer. The model must learn that this token means “the key ends here and the value begins” as an emergent property of training, not as an explicit structural signal.

Property	Pipe-delimited	JSON
Grammar tokens isolated	99.5%	7.5%
Grammar fused with grammar	0%	92.5% (43/43 tokenizers)
Grammar is deterministic	Yes	No

4.9 Attention Mechanism Analysis

Parts 4.1-4.8 operate at the tokenizer level. This section examines what happens inside the transformer when it processes merged vs. clean token sequences.

We loaded Pythia 410M (24 layers, 16 heads) and Gemma 2B (26 layers, 8 heads) and extracted attention weights from every layer and head while processing identical data in pipe-delimited vs JSON format at increasing scale.

Attention entropy crossover Attention entropy measures how spread out the model's attention is. High entropy means attention is distributed uniformly (the model is looking everywhere, finding nothing). Low entropy means attention is focused.

Pythia 410M:

Orders	Pipe-delimited entropy	JSON entropy	Delta
5	3.03	2.87	JSON lower (model knows JSON)
10	3.32	3.01	JSON still lower
20	3.66	3.16	JSON still lower
50	**3.99**	**4.50**	**JSON crosses over (+13%)**

At small scale, JSON entropy is lower. The model has been trained on billions of JSON examples and has learned efficient attention patterns. At 50 orders, the crossover: JSON entropy exceeds the comparison format by 13%. The model's learned JSON parsing breaks down. The repeated field names produce thousands of identical token IDs competing for attention. The model can no longer distinguish them using positional encoding alone.

Grammar attention collapse We classified every token as grammar or payload and measured attention allocation.

Gemma 2B:

Orders	Pipe grammar%	Pipe payload%	JSON grammar%	JSON payload%
5	53.8%	46.2%	29.8%	67.7%
10	52.4%	47.6%	29.7%	68.0%
20	50.8%	49.2%	30.4%	67.4%
50	**48.6%**	**51.4%**	**8.6%**	**86.3%**
100	**37.4%**	**62.6%**	**8.6%**	**86.3%**

At small scale, JSON's attention splits roughly 30% grammar / 68% payload. The model attends to structural tokens to understand the format, then to payload tokens to extract data. At 50 orders, JSON grammar attention collapses from 30% to 8.6%. The model stops attending to structural tokens. It is no longer parsing JSON's structure; it distributes attention uniformly across content, unable to distinguish structure from data.

The pipe-delimited format does the opposite. Payload attention increases steadily from 46% to 63% at 100 orders. As the payload grows, the model progressively focuses more on data and less on grammar. The grammar is proportionally smaller (one header vs hundreds of rows) and every grammar token is unambiguous.

Token repetition scaling The underlying cause of both phenomena is token repetition. JSON repeats the same token IDs on every row. The pipe-delimited format does not.

Orders	Pipe-delimited repeat%	JSON repeat%	Tokens
5	42.4%	74.8%	110 / 245
10	62.0%	84.9%	211 / 490
20	76.6%	91.0%	413 / 982
50	83.6%	93.6%	1,019 / 2,048

At 50 orders, 93.6% of JSON tokens are duplicates. The model's attention mechanism must distribute weight across hundreds of identical tokens using only positional encoding to distinguish them. This is mathematically equivalent to the attention dilution problem described by Ildiz et al. (2024): self-attention weights tokens proportionally to their frequency, so repeated tokens dominate the attention budget.

This is the transformer-level mechanism behind comprehension failure: the model's attention cannot sustain structural parsing at scale when boundaries are hidden in merged tokens and the majority of the sequence is repetitive noise.

---

5. Discussion

5.1 The Training Familiarity Paradox

The conventional wisdom holds that LLMs "know" JSON best because they have been trained on more JSON than any other structured format. This is true at the model level: transformer weights have learned JSON's semantics from billions of examples. But at the tokenizer level, the opposite occurs. The more JSON the tokenizer saw during training, the more aggressively it merged JSON patterns, and the more structural boundaries it hid.

The models that saw the most JSON have the worst JSON boundaries:

- GPT-4 (massive code corpus): 117 merged quote+field entries
- LLaMA 3 (large code mix): 153 merged entries
- Qwen 2.5 (heavy code training): 154 merged entries
- Claude (different tokenizer strategy): 3 merged entries
- Gemma 2/3 (different merge policy): 2-4 merged entries

Training familiarity did not create structural understanding. It created compression. The tokenizer optimized for representing JSON in fewer tokens, which is exactly what a compression algorithm should do. But compression hides structure. The quote and the field name became one token because that is more efficient for storage. It is less efficient for comprehension.

This inverts the standard argument. "Trained on JSON" is not an advantage for structural comprehension at scale. It is the mechanism that causes structural ambiguity.

5.2 Why Claude and Gemma Differ

Claude's tokenizer has 3 quote+letter entries. Gemma 2 has 4. Gemma 3 has 2. Across the 43 tokenizers tested, 11 tokenizers have zero or near-zero quote merge entries. The specific training details are proprietary, but measurable differences suggest explanations:

- **Vocabulary size**: Claude uses approximately 65,000 entries (one of the smallest tested). Smaller vocabularies are more conservative about which merges to include. But Gemma 3's vocabulary is the largest tested (262,000 entries) yet has only 2 quote merges. Vocabulary size alone does not predict merge behavior.
- **Training data mix**: Tokenizers trained on corpora with less code/JSON relative to natural language will see `"name` less frequently, making it less likely to cross the merge threshold.
- **Merge boundary policy**: BPE training can be configured to treat certain characters as merge barriers (never merge across them). Anthropic and Google may have intentionally prevented `"` from merging with adjacent letters.

The merge policy matters more than vocabulary size. This suggests that structure-aware tokenizer design, where delimiter characters are treated as merge barriers during BPE training, could eliminate boundary merging entirely.

5.3 Irrecoverability

Four properties make this problem unfixable for existing models:

1. **Vocabulary is frozen.** Once the tokenizer is trained, its vocabulary never changes. Model fine-tuning adjusts weights but cannot add, remove, or modify vocabulary entries.
2. **All weights depend on the vocabulary.** Token ID #32586 has a learned embedding vector in GPT-4's weights. Removing it would break every layer that references that embedding.
3. **Tokenization is pre-model.** The merge happens before the transformer processes the input. By the time the model sees the sequence, `"name` is already a single integer. The model cannot see inside a token to decompose it.

4. **Retraining the tokenizer requires retraining the model.** A new vocabulary means new token IDs, new embeddings, new attention patterns. The entire model must be retrained from scratch.

No amount of prompt engineering, fine-tuning, or reinforcement learning from human feedback can change the fact that `"name` is token #32586 in GPT-4's dictionary. The only mitigation is to use a format whose grammar characters do not appear as merged entries in tokenizer vocabularies.

5.4 Comprehension Correlation

The tokenization analysis connects to observed comprehension outcomes from 2,400+ LLM evaluations across 11 models:

- JSON accuracy on stress-scale data (500 records): 53.4% (10 models, 24 runs)
- GCF accuracy on stress-scale data: 91.2%
- GCF accuracy on standard workloads: 100% on every frontier model

The failure mechanism has three compounding components:

1. **Ambiguous structural boundaries** (8.93% merge rate). The model sees different structure depending on which tokenizer it uses.
2. **Overwhelming repetition** (52% of tokens are repeated field names). The attention mechanism has 5,500 tokens that all look identical competing for attention budget.
3. **Low signal-to-noise ratio** (19% data, 81% overhead). The model must find relevant information buried in structural noise.

Ildiz et al. (2024) proved mathematically that self-attention weights tokens proportionally to their frequency in the input sequence. Their Context-Conditioned Markov Chain (CCMC) formulation shows that P(next_token = j | X) includes m_j (the count of token j) in the numerator. Tokens appearing more frequently receive more attention weight purely by count. In a 500-row JSON array, structural tokens account for ~80% of occurrences, meaning the model's attention budget is mathematically dominated by semantically redundant content, while data values (numerically outnumbered) receive proportionally less attention. The paper analyzes single-layer models; multi-layer architectures partially mitigate this, but our comprehension data shows the mitigation is insufficient at 500+ rows.

Observed failure patterns are consistent with this mechanism:

Pattern	Example	Tokenization Explanation
Deterministic miscounts	GPT-5.4 always answers `edge_count=198` (correct: 200)	Consistent with cl100k/o200k merge patterns on every row

Pattern	Example	Tokenization Explanation
Context overwhelm	GPT-5.5 returns empty string on most questions	53K tokens, 81% overhead; attention has nothing to lock onto
Large counting errors	Distance-filtering wrong by 50-140	Must attend to 500 identical `"distance":` patterns

GCF's errors are small (off by 1-2) because the model understood the structure but slightly misread a value. JSON's errors are large (off by 50-140) because the model could not find the structure at all. Error magnitude data: GCF median error 4, JSON median error 56.

5.5 TOON's Tab Delimiter Is Worse Than JSON's Quote

TOON, the primary competing token-efficient format, uses tab characters as column delimiters. We applied the same analysis to TOON's grammar symbols. Tab merges with adjacent content more aggressively than JSON's quote character:

Format	Delimiter	Merge Rate	Total Checks
TOON	Tab	**32.91%**	860
JSON	Quote	8.17%	1,935
GCF	Pipe	**0.47%**	29,025

The exhaustive vocabulary dump explains why. GPT-4o has a **100% tab merge rate**: every single word tested merges with the preceding tab. GPT-4 cl100k merges 95%. The tab has **1,238 unique mergeable words** across all 43 tokenizer vocabularies. JSON's combined surface is 707 words (quote 193, colon 232, comma 282). The pipe has 24. Tab-separated data (TSV, log files, shell output) is so common in code training corpora that nearly every common word is fused with the tab character in OpenAI's vocabularies.

GPT-4 cl100k has **1,173 tab+letter vocabulary entries** and 22 pipe+letter entries. GPT-4o o200k has **1,036 tab+letter** entries and 6 pipe+letter entries. No other tokenizer family has tab+letter entries.

The training familiarity paradox applies doubly to TOON: its chosen delimiter (tab) is the most over-represented structural character in code training data, producing the highest merge rates of any format tested. TOON chose the delimiter with the largest adversarial surface of any common separator character (52x larger than the pipe).

5.6 Why Merging Causes Higher Degradation at Scale

At 10 rows, `"name` being one token instead of two does not matter. The model has seen enough JSON to handle it. There are only 10 merged boundaries. The attention mechanism can work around it.

At 500 rows, three problems compound simultaneously:

1. The merged boundary repeats 500 times. Each row contains `"name":`, `"id":`, `"type":`. That creates approximately 1,500 positions where the structural boundary is inside a merged token. The model must decompose structure from inside merged tokens at 1,500 positions, not 10.

2. All 1,500 positions are identical token sequences. The token for `"name` on row 1 is the same integer (#32586) as on row 500. The model cannot distinguish them. It relies on positional encoding alone to track "which `"name` am I looking at?" Positional encoding degrades over long sequences.

3. 81% of the sequence is noise. The repeated field names and braces are not just merged; they are also redundant. The attention mechanism is spread across approximately 8,500 tokens that carry no information, trying to find the approximately 2,000 tokens that do. The merged boundaries make the noise harder to skip because the model cannot cleanly identify where structure ends and data begins.

The compounding is critical. At 10 rows: 10 merged boundaries, small attention budget, manageable. At 500 rows: 1,500 merged boundaries, massive noise, positional encoding stretched to capacity, attention diluted across thousands of identical tokens. The model stops trying to find precise answers and guesses. This is why JSON errors at scale are off by 50-140 (comprehension failure), not off by 1-2 (precision error). The model did not slightly misread a number. It could not find the answer at all.

GCF at 500 rows: zero merged boundaries on field names, 99.8% signal, and structure answers questions directly (e.g., `## related [167]` encodes the count in the section header). Nothing compounds because there is no ambiguity, no repetition, and no noise to dilute attention.

5.7 Attention Dilution in Detail

Self-attention allocates a fixed budget across all input positions. When a query token looks for relevant keys, it must attend over the entire sequence. When 80% of that sequence is structural noise, the budget is diluted across positions that carry no information.

Consider the task "how many records have status = shipped?" given 500 JSON objects. The model must attend to every `"status":` pattern (500 occurrences), read the following value, compare to "shipped," and count matches. The 500 `"status":` patterns produce the same tokens every time. The model has no structural marker distinguishing the 150th occurrence from the 350th.

Ildiz et al. (2024) proved that self-attention weights tokens proportionally to their frequency (the CCMC formula includes m_j in the numerator). When 80% of the sequence is repetitive structural tokens, the formula guarantees those tokens dominate the attention budget by count, leaving proportionally less for the data values.

In a header-factored format with pipe delimiters, the equivalent task requires attending to a column of values at known, consistent positions. No ambiguity. No repetition competing for attention. The structural delimiter (pipe) is always at the same relative position within each row.

Our attention mechanism analysis (Section 4.9) provides direct empirical evidence for this theoretical prediction. Gemma 2B's grammar attention drops from 30% to 8.6% at 50 orders, confirming that

the model abandons structural parsing at exactly the scale where attention dilution theory predicts failure. The entropy crossover at the same scale (Section 4.9) confirms the mechanism from a complementary angle: the model's attention distribution becomes more uniform on JSON than on the comparison format precisely when repetitive structural tokens begin to dominate.

5.8 Implications for Tokenizer Design

The results in this paper point to a design principle: BPE tokenizer training should treat structural delimiter characters as merge barriers. A tokenizer that never merges quotes, colons, tabs, pipes, or braces with adjacent content would produce clean structural boundaries by construction, regardless of vocabulary size or training corpus.

The evidence that this is feasible comes from Claude and Gemma. Both have near-zero quote merge entries despite being trained on large corpora that include JSON. Whether this was intentional policy or an artifact of their training configuration, the result demonstrates that clean boundaries are achievable without sacrificing general-purpose tokenization quality.

Future tokenizer designs could formalize this principle: define a set of structural characters and enforce that they never participate in BPE merges. The resulting tokenizer would produce slightly more tokens on structured data (each delimiter is its own token rather than merging into adjacent content), but every model using that tokenizer would see consistent, unambiguous structural boundaries. The tradeoff between compression and comprehension would shift explicitly in favor of comprehension for structured workloads.

---

6. Related Work

Deekeswar (2024) measured that 1,000 JSON records consume approximately 80,000 tokens with the majority being repeated keys and punctuation, proposing ONTO as an alternative format. Our analysis explains the mechanism: 52% of tokens are repeated field names, and these names fuse with structural delimiters on half of tokenizers.

Kutschka and Geiger (2026) found that token-efficient formats can hurt accuracy in some configurations, arguing that training distribution favoring JSON compensates for inefficiency. Our data partially confirms this at small scale (all formats achieve near-100% at 10-50 records) but shows the compensation fails at 500+ records where JSON drops to 53.4%.

Ildiz et al. (2024) proved that self-attention implements a Context-Conditioned Markov Chain where the probability of attending to token j includes m_j (its frequency in the sequence) in the numerator. This is the mathematical basis for our attention dilution finding: when structural tokens like `"name":` account for 80% of occurrences in a JSON array, they dominate the attention budget by count. The paper analyzes single-layer models; our comprehension data confirms the effect persists in production multi-layer architectures at 500+ rows.

Karim and Batatia (2025) proposed using fixed tokens for structure and BPE for values. GCF achieves a similar result through grammar design: choosing structural characters from the set that BPE tokenizers never merge with adjacent content.

Sui et al. (2023) showed that table format affects LLM performance across multiple tasks. Our analysis explains this finding at the BPE level: different formats produce different merge patterns, different overhead ratios, and different signal-to-noise ratios.

Matveev (2026) argued that JSON's advantage from training distribution scales with data complexity, proposing that alternative formats only separate past a complexity threshold. Our evaluation data confirms the threshold exists at approximately 100-200 records for nested data and approximately 500 for flat tables.

Liyanage and Yvon (2025) studied post-training tokenizer adaptation, demonstrating that tokenizer changes after model training degrade performance. This supports our irrecoverability argument: modifying a tokenizer's vocabulary to fix JSON merge patterns would require retraining the model from scratch.

Web Data Commons (University of Mannheim, 2024) analyzed JSON properties across 2.39 billion web pages. The two most common properties, `name` (3.5 billion occurrences) and `url` (2.6 billion occurrences), are both among our 15 worst-offending merge patterns. This confirms that the affected field names are not edge cases but represent the most frequent JSON patterns on the web.

---

7. Conclusion

JSON's structural grammar creates hidden boundaries in BPE tokenizer vocabularies. The opening quote character merges with common field names into single tokens on 4-5 of 8 tested tokenizers. These merges are hardcoded vocabulary entries (dictionary lookups, not context-dependent decisions), and they are irrecoverable without retraining both the tokenizer and the model.

The problem compounds at scale. Each merged field name repeats on every row, so 500 rows produce approximately 1,500 hidden boundaries for field names like `id`, `name`, and `type`. Combined with 81% token overhead (52% repeated field names, 29% structural characters), this creates a dual failure mode: ambiguous boundaries dilute attention while repetitive noise consumes the attention budget.

The findings are format-agnostic in implication. Any format designer can apply the principles demonstrated here: choose structural delimiters from the set of characters with the lowest merge rates across tokenizers, declare field names once rather than repeating them per row, and encode values positionally to eliminate key-value pair overhead. Our grammar swap experiment confirms these structural choices produce consistent savings (0.4 percentage point spread across 800 measurements) regardless of which specific delimiter characters are used.

Three properties make this analysis actionable:

1. The problem is measurable. Boundary merge rates, vocabulary entry counts, and overhead percentages can be computed for any format on any tokenizer.
2. The problem is deterministic. A vocabulary entry either exists or it does not. The merge either always occurs or never occurs for a given field name on a given tokenizer.
3. The problem is permanent for existing models. Frozen vocabularies cannot be modified post-training.

Future work should investigate three directions: (1) causal relationships between boundary merges and comprehension errors through controlled experiments with structure-aware tokenizers that enforce merge barriers on delimiter characters, (2) whether models trained with barrier-aware tokenizers exhibit faster convergence on structured data tasks compared to models trained with standard BPE, and (3) optimal grammar search algorithms that minimize total tokens while maximizing boundary consistency for a given vocabulary. The attention mechanism analysis in this paper (Section 4.9) provides the first empirical evidence connecting tokenizer-level merges to transformer-level attention failure, establishing the mechanistic link that future causal studies should exploit.

---

8. Reproducibility

All experiments are reproducible. The analysis scripts are open source and require only the GCF library and tokenizer packages.

Dependencies

Primary analysis (Python):

```
tokenizers
huggingface_hub
tiktoken
torch              # attention analysis only
transformers       # attention analysis only
```

Supplementary analysis (JavaScript):

```
@blackwell-systems/gcf
@lenml/tokenizers
@lenml/tokenizer-claude
@lenml/tokenizer-gpt4
@lenml/tokenizer-gpt4o
@lenml/tokenizer-llama3_1
@lenml/tokenizer-qwen2_5
@lenml/tokenizer-deepseek_v3
```

```
@lenml/tokenizer-gemma2
@lenml/tokenizer-mistral_nemo
```

Scripts

Primary analysis (43 tokenizers, Python)

Script	Purpose	Key Output
`hf-tokenizer-analysis.py`	43-tokenizer merge rates, vocab entries, savings vs JSON and TOON	Comprehensive cross-tokenizer tables
`structural-equivalence-proof.py`	Grammar isolation across 43 tokenizers	99.5% isolation proof
`adversarial-vocab-dump.py`	Exhaustive vocabulary scan, complete adversarial surface	Pipe 24 vs JSON combined 707 vs tab 1,238
`attention-analysis.py`	Attention extraction from Pythia 410M and Gemma 2B	Entropy crossover, grammar attention collapse

Supplementary analyses (8 tokenizers, JavaScript)

Script	Purpose	Key Output
`tokenizer-variance.mjs`	Token savings consistency across 8 tokenizers, multiple scales	Per-tokenizer savings table
`structural-variance.mjs`	Boundary merge analysis, structural consistency	Merge rate comparison
`common-field-merge-analysis.mjs`	155 common field names, merge rates per tokenizer	Worst-offending fields
`json-tokenization-analysis.mjs`	Token distribution breakdown, overhead scaling	Overhead percentages by category
`worst-json-tokenization.mjs`	Maximum tokenization variance search (840 patterns)	7-way tokenization example
`exhaustive-json-boundary-search.mjs`	Exhaustive boundary search (8,434 patterns)	Complete merge pattern catalog

Script	Purpose	Key Output
`graph-token-efficiency.mjs`	Graph profile savings across tokenizers	63-69% savings range
`session-dedup-efficiency.mjs`	Session deduplication savings	84-92% cumulative savings
`tokenizer-vocabulary-analysis.mjs`	Vocabulary entry lookup for specific merged tokens	Token IDs per field per tokenizer
`vocabulary-full-scan.mjs`	Exhaustive full vocabulary scan, all 8 tokenizers	Quote+letter and pipe+letter entry counts
`toon-tokenizer-analysis.mjs`	TOON tab delimiter merge analysis	Tab vs quote vs pipe merge rates
`grammar-swap-experiment.mjs`	5 delimiter sets, 800 measurements	0.4pp spread proving structural savings

Repository: github.com/blackwell-systems/gcf

Appendix A: Syntactic Deep Dive

For completeness, we provide exact tokenization of representative structured data patterns across tokenizers. This data demonstrates that grammar symbol variance is limited to payload content, not structural boundaries.

A.1 Edge declaration: `@0<@2|implements`

Tokenizer	Tokens	Split
Claude	7	`@ 0 < @ 2 | implements`
GPT-4	7	`@ 0 < @ 2 | implements`
GPT-4o	7	`@ 0 < @ 2 | implements`
LLaMA 3.1	7	`@ 0 < @ 2 | implements`
Qwen 2.5	7	`@ 0 < @ 2 | implements`
DeepSeek V3	8	`@ 0 < @ 2 | im plements`
Gemma 2	7	`@ 0 < @ 2 | implements`

Tokenizer	Tokens	Split
Mistral Nemo	8	`@ 0 < @ 2 | im plements`

All structural characters (`@`, `<`, `|`) are always single tokens. The only variance is in the value `implements` (1 vs 2 tokens), which does not affect parsing.

A.2 Symbol row: `@0|function|auth.validateToken|0.95|definition`

Tokenizer	Tokens	Key Differences
GPT-4	14	Merges `.validate` (1 tok), `95` (1 tok)
Qwen 2.5	15	Splits `95` into `9` + `5`
Gemma 2	16	Splits `.` + `validate`, splits `9` + `5`

Pipe delimiters are always single tokens across all tokenizers. Variance is only in how tokenizers handle value content: dot-prefixed words (`.validate` as 1 tok on GPT-4 vs `.` + `validate` on Gemma) and two-digit numbers (`95` as 1 tok on GPT-4 vs `9` + `5` on Qwen/Gemma). This is value variance (harmless) not boundary variance (dangerous).

A.3 Delimiter selection rationale

From the 74 safe ASCII characters, structural delimiters were chosen based on semantics and readability:

Character	Why chosen	Alternative considered	Why not
`|` (pipe)	Rare in natural text, visually distinct column separator	Tab (`\t`)	Invisible, merges on 33% of tokenizers
`@`	Establishes “this is an ID” semantically	`$`	Also safe, but less intuitive

Character	Why chosen	Alternative considered	Why not
##	Two-char sequence tokenizers always merge into one token, Markdown-familiar	===	3 chars, less efficient
<	Reads as “points to” for edges	~	Also safe, but less semantic
\n	Universal row separator, zero overhead	;	Less readable
,	Schema field separator, familiar from CSV	:	Conflicts with potential value content

---

References

Blackwell, D. (2026). GCF: A Token-Optimized Wire Format for Structured LLM Interactions. DOI: 10.5281/zenodo.20579817.

Deekeswar, H. (2024). ONTO: A Token-Efficient Columnar Notation for LLM Input Optimization. arXiv:2604.17512.

Ildiz, M. E., Huang, Y., Li, Y., Rawat, A. S., & Oymak, S. (2024). From Self-Attention to Markov Models: Unveiling the Dynamics of Generative Transformers. arXiv:2402.13512.

Karim, K. & Batatia, H. (2025). Innovative Tokenisation of Structured Data for LLM Training. arXiv:2508.01685.

Kutschka, L. & Geiger, B. (2026). Notation Matters: A Benchmark Study of Token-Optimized Formats in Agentic AI Systems. arXiv:2605.29676.

Liyanage, V. & Yvon, F. (2025). AdaptBPE: From General Purpose to Specialized Tokenizers. arXiv:2601.21665.

Matveev, I. (2026). Token-Oriented Object Notation vs JSON: A Benchmark of Plain and Constrained Decoding Generation. arXiv:2603.03306.

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. In Proceedings of the 54th Annual Meeting of the ACL (pp. 1715-1725).

Sui, Y., He, M., Zhang, Z., Wang, Y., & Zhao, J. (2023). Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study. arXiv:2305.13062.

University of Mannheim. (2024). Web Data Commons: RDFa, Microdata, and Microformat Data Sets. http://webdatacommons.org/structureddata/

Corresponding author: Dayna Blackwell, Blackwell Systems (dayna@blackwell-systems.com)